



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación

Materia:
Sistemas Operativos

**SISTEMAS EMPOTRADOS
Y DE TIEMPO REAL**

INTEGRANTES

Díaz Pontón Steven Santiago
Gamarra Zárate Jamileth Estefanía
Herrera Ramos Thais Melanie
Tigasi Sampetro Paul Alexander
Trujillo Vega Mayummy Jailly

Quevedo, Ecuador · 2026

1. Introducción

Los sistemas empotrados forman parte de nuestra vida diaria. Están dentro de aparatos como electrodomésticos, ascensores, automóviles, teléfonos, equipos médicos y semáforos, y les permiten realizar tareas específicas de manera automática. El microondas cuenta el tiempo y apaga el calentamiento; la impresora mueve el papel y controla la tinta; el semáforo cambia las luces en un orden establecido; una alarma detecta movimiento y activa una sirena.

Estos sistemas combinan una parte física compuesta por placa, memoria, sensores y conexiones, con un programa que les indica qué hacer. En muchos casos, además, deben responder dentro de un plazo determinado para que su función tenga sentido: de ahí la estrecha relación con los sistemas de tiempo real, que esta consulta también explica.

2. ¿Qué es un sistema empotrado?

Un sistema empotrado (o sistema embebido) es un sistema informático de uso específico formado por hardware (componentes físicos) y software (programas), diseñado para realizar una función determinada o un conjunto reducido de funciones. Se llama empotrado porque forma parte de un sistema más grande, como un automóvil, una lavadora o un teléfono inteligente.

IBM (International Business Machines) lo describe como un sistema pensado para administrar el hardware de dispositivos especializados, a diferencia de un sistema operativo de propósito general [1]. Por ejemplo, el sistema que controla un **cajero automático (ATM)** solo procesa retiros, depósitos y consultas de saldo: no puede usarse para navegar en internet.

En una lavadora, el sistema empotrado recibe la opción elegida, mide el nivel de agua y decide cuándo debe girar el motor. En un ascensor, detecta el piso solicitado, abre las puertas y evita que se mueva si una puerta está abierta.

2.1 Características principales

Característica	¿Qué significa?	Ejemplo
Función dedicada	Hace una sola tarea; no se le instalan programas nuevos	Marcapasos: solo regula el corazón
Recursos limitados	Poca memoria y procesador restringidos	Sensor de temperatura simple
Bajo consumo	Clave en dispositivos portátiles o con batería	Reloj inteligente (wearable, dispositivo vestible)
Tiempo real (a veces)	Debe responder dentro de un plazo definido	Airbag (bolsa de aire) de un automóvil
Fiabilidad	Funciona sin fallos, muchas veces sin ayuda humana	Controlador industrial 24/7
Tamaño reducido	Integración compacta, a veces en un solo chip	Wearables (dispositivos vestibles), sensores IoT

Tabla 1. Características principales de un sistema empotrado.

3. Arquitectura de un sistema empujado

Un sistema empujado combina componentes de hardware, como la unidad de procesamiento (microcontrolador o microprocesador), memoria, sensores, actuadores y fuente de alimentación, con capas de software que van desde los drivers (controladores) que controlan el hardware hasta la aplicación final. Cuando el aparato realiza varias tareas a la vez, puede apoyarse en un sistema operativo de tiempo real (RTOS).

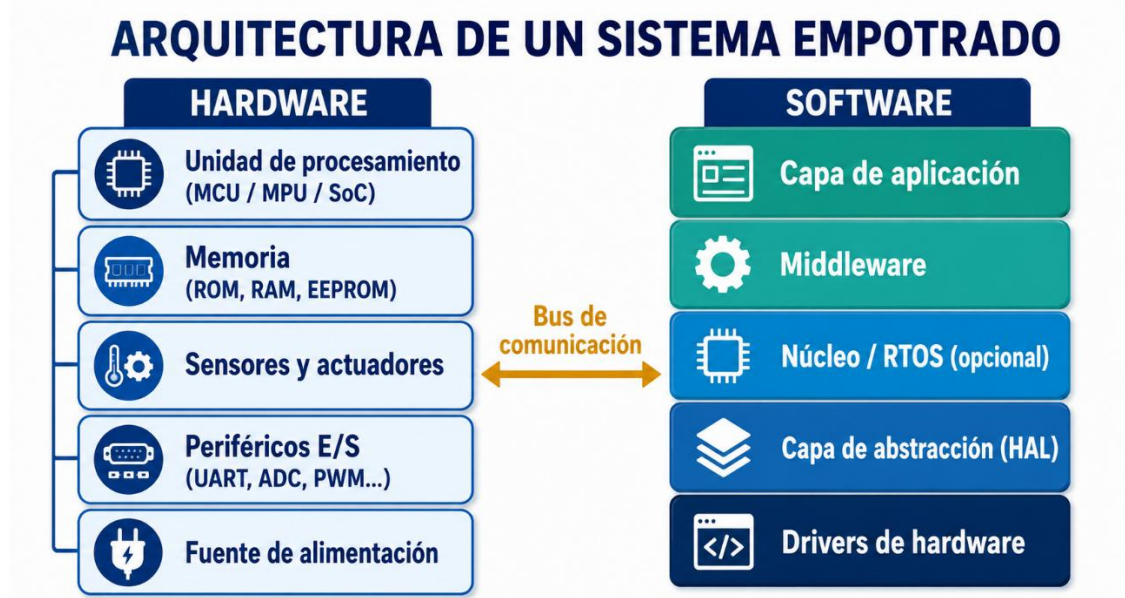


Figura 1. Arquitectura típica de un sistema empujado.

Forma de trabajo	¿Cómo funciona?	Ventaja	Ejemplo
Sin sistema operativo	El programa trabaja directamente en la placa	Sencillo, usa poca memoria	Sensor de luz
Con RTOS	Un sistema especial organiza varias tareas	Atiende primero lo urgente	Dron o máquina industrial
Sistema más completo	Incluye conexión y almacenamiento	Aplicaciones más avanzadas	Cámara inteligente

Tabla 2. Formas comunes de ejecutar el programa de un sistema empujado.

Cuando el dispositivo no utiliza sistema operativo, el desarrollo se conoce como programación bare-metal (directamente sobre el hardware, sin sistema operativo): el firmware accede directamente a los registros del microcontrolador, sin capas intermedias que faciliten la portabilidad, pero con un control total sobre el hardware [9].

4. El firmware: creación y actualización

El programa interno que controla el aparato se llama firmware. Primero existe una base con funciones generales (leer un botón, recibir una señal); si el fabricante necesita algo adicional, crea un programa propio, generalmente en C o C++. Al compilar, ambas partes se unen en un solo software que se graba en la memoria de la placa.

En dispositivos sencillos (sensores, controles remotos) el firmware se graba una sola vez. En equipos modernos (televisores, automóviles, relojes) puede actualizarse: el dispositivo comprueba que la actualización viene realmente del fabricante y, en muchos casos, conserva la versión anterior por si la nueva falla. A este mecanismo de actualización remota se le llama **OTA (Over-The-Air, actualización por el aire)** [2], [3].

5. Clasificación y aplicaciones de los sistemas empotrados

Los sistemas empotrados se pueden clasificar según su complejidad (desde un solo microcontrolador hasta sistemas con Linux embebido), según su conectividad (autónomos o en red, como en el IoT) y según si tienen o no restricciones de tiempo real. Se usan prácticamente en todos los sectores:

Sector	Ejemplo concreto
Hogar	Microondas, lavadora, control remoto de TV
Banca	Cajero automático (ATM)
Medicina	Marcapasos, bombas de insulina
Industria	Controladores lógicos (PLC, Programmable Logic Controller), robots
Automoción	Unidad de control del motor (ECU, Electronic Control Unit), ABS (frenos antibloqueo), airbags
Aeroespacial	Sistemas de control de vuelo, CubeSats
Consumo	Smartphones, routers, cámaras, wearables

Tabla 3. Sistemas empotrados en distintos sectores.

En el sector aeroespacial, agencias como la NASA (National Aeronautics and Space Administration, Administración Nacional de Aeronáutica y del Espacio) y la Agencia Espacial Europea (ESA) emplean sistemas empotrados en pequeños satélites o CubeSats para tareas de control, comunicación y observación [10]. En el sector médico, dispositivos como marcapasos y bombas de insulina se clasifican regulatoriamente como software integrado en un dispositivo médico, sujeto a validación por parte de organismos como la FDA (Food and Drug Administration, Administración de Alimentos y Medicamentos de EE. UU.) [11]. Gran parte de estos sistemas se construyen sobre microcontroladores comerciales, cuyos fabricantes como Texas Instruments documentan públicamente sus familias de productos y aplicaciones [12].

6. Sistemas de tiempo real

Un sistema de tiempo real (STR) es aquel cuya corrección depende no solo del resultado, sino también del instante en que ese resultado se produce. No basta con que la respuesta sea correcta: si llega tarde, se considera una falla, aunque el cálculo en sí esté bien hecho.

Un ejemplo simple: un robot que debe tomar una pieza de una banda transportadora. Si llega tarde, la pieza ya no está donde debía estar, aunque el robot haya llegado al lugar correcto. Lo mismo ocurre con un airbag que se activa varios segundos después de un choque, o un freno ABS que evita que las ruedas se bloqueen.

6.1 Tipos según la importancia del tiempo

CLASIFICACIÓN DE LOS SISTEMAS DE TIEMPO REAL

Se clasifican según qué ocurre si la respuesta no llega dentro del plazo



Figura 2. Clasificación de los sistemas de tiempo real según la consecuencia del retraso.

En el tiempo real duro, responder tarde puede provocar un daño grave (airbag, marcapasos, frenos ABS Anti-lock Braking System, sistema de frenos antibloqueo). En el tiempo real firme, la respuesta tardía ya no resulta útil y se descarta, pero el sistema sigue funcionando (una cámara industrial que detecta tarde un defecto). En el tiempo real blando, un retraso no produce peligro, solo empeora el servicio (un video que se congela, una videollamada con demora).

7. Relación entre sistemas empotrados y sistemas de tiempo real

Los dos términos no significan exactamente lo mismo, aunque suelen aparecer juntos. Un sistema empotrado se define por estar integrado dentro de un aparato y realizar una función específica. Un sistema de tiempo real se define por tener que responder antes de un plazo. Muchos sistemas de tiempo real también son empotrados como un airbag o un marcapasos; sin embargo, una cafetera sencilla puede ser empotrada sin necesitar un tiempo de respuesta tan estricto.

Aspecto	Sistema empotrado	Sistema de tiempo real
Objetivo	Controlar una función concreta dentro de un aparato	Responder correctamente antes de un tiempo límite
Pregunta principal	¿Qué función realiza el dispositivo?	¿La respuesta llega en el momento requerido?
Ejemplo	Control remoto, lavadora, proyector	Airbag, frenos ABS, marcapasos
Relación	Puede trabajar sin requisitos estrictos de tiempo	Con frecuencia está implementado dentro de un sistema empotrado

Tabla 4. Diferencias y relación entre sistemas empotrados y sistemas de tiempo real.

8. Sistemas operativos de tiempo real (RTOS)

Cuando un sistema empujado necesita ejecutar varias tareas a la vez y cumplir plazos, se usa un **RTOS (Real-Time Operating System, sistema operativo de tiempo real)**: un sistema operativo diseñado para dar respuestas predecibles, priorizando la previsibilidad sobre el rendimiento medio. IBM explica que, a diferencia de Windows o Linux (pensados para eficiencia general), un RTOS se enfoca en dar respuestas a tiempo, sobre todo a las tareas más críticas [2].

Aspecto	RTOS	SO de propósito general
Objetivo principal	Previsibilidad / determinismo	Máximo rendimiento medio
Planificación	Basada en prioridades	Basada en equidad (fairness)
Tamaño del núcleo	Pequeño (kilobytes)	Grande (megabytes)
Ejemplos	FreeRTOS, VxWorks, QNX, Zephyr	Windows, Linux estándar, macOS

Tabla 5. RTOS frente a sistema operativo de propósito general.

FreeRTOS es un RTOS de código abierto muy usado en microcontroladores; Amazon, que hoy lo mantiene, lo describe como el sistema operativo de tiempo real líder del mercado para microcontroladores [5]. La NASA reporta su uso en pequeños satélites (*CubeSats*), por ser liviano y funcionar con poca memoria disponible a bordo [4]. Otros ejemplos conocidos son VxWorks (aeronáutica y misiones espaciales), QNX (automoción) y Zephyr, impulsado por la Linux Foundation para proyectos de IoT [6].

Dos aparatos empujados pueden tener programas distintos y aun así comunicarse: el control remoto del aire acondicionado tiene su propio programa y el equipo otro; al presionar un botón, el control envía una señal que el equipo interpreta. Lo mismo ocurre con un reloj que envía datos al teléfono por Bluetooth.

9. Tecnologías relacionadas: IoT, edge computing y TinyML

Cuando un aparato se conecta a Internet para enviar información o recibir órdenes, forma parte del Internet de las Cosas (IoT): un termostato controlado desde el celular, una cámara de seguridad que envía avisos, o un reloj que registra la actividad física.

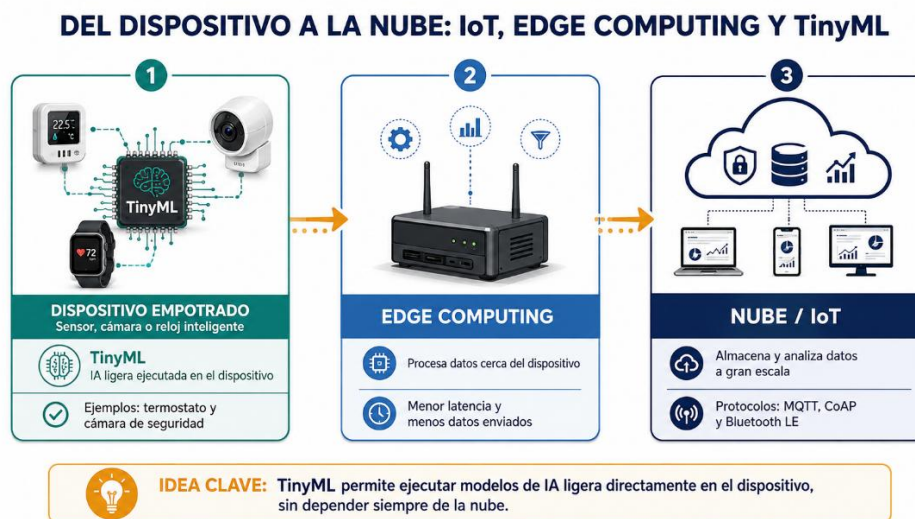


Figura 3. Del dispositivo a la nube: IoT, edge computing y TinyML.

El *edge computing* (computación en el borde) procesa los datos cerca del dispositivo, reduciendo la latencia frente a enviarlo todo a la nube. El **TinyML** (aprendizaje automático diminuto/ligero) permite ejecutar modelos

de inteligencia artificial ligeros directamente en microcontroladores pequeños, por ejemplo para reconocer una palabra o detectar un ruido extraño en una máquina [7].

10. Ventajas, desventajas y retos actuales

- Ventajas: alta eficiencia, bajo consumo energético, respuesta predecible, costo reducido en producción masiva.
- Desventajas: recursos de hardware muy restringidos, complejidad en el desarrollo, dificultad para actualizar algunos dispositivos.
- Retos actuales: garantizar la ciberseguridad sin comprometer los recursos limitados, escalar frente al crecimiento del IoT, y diseñar mecanismos confiables de actualización remota (OTA) [8].

11. Conclusiones

- Un sistema empotrado es una pequeña computadora integrada en la placa de un aparato para controlar una función específica; combina hardware dedicado con firmware hecho a la medida.
- El firmware puede grabarse una sola vez (dispositivos simples) o ser actualizable (dispositivos modernos conectados).
- Un sistema de tiempo real no se define por su velocidad, sino por su capacidad de responder dentro de un plazo: una respuesta correcta pero tardía se considera una falla.
- No todo sistema empotrado es de tiempo real, y no todo sistema de tiempo real es empotrado; sin embargo, ambos conceptos se combinan con frecuencia en dispositivos críticos como airbags o marcapasos.
- La conexión a Internet (IoT), el procesamiento en el borde (edge computing) y la IA ligera (TinyML) amplían las capacidades de estos sistemas, pero también exigen más seguridad y mantenimiento.

12. Referencias

- [1] IBM, “What is an operating system?,” IBM Think, 2025. Disponible: <https://www.ibm.com/think/topics/operating-systems>
- [2] IBM, “What is a real-time operating system (RTOS)?,” IBM Think, 2025. Disponible: <https://www.ibm.com/think/topics/real-time-operating-system>
- [3] NIST, “IoT device cybersecurity: Software update,” National Institute of Standards and Technology. Disponible: <https://pages.nist.gov/IoT-Device-Cybersecurity-Requirement-Catalogs/technical/update/>
- [4] NASA, “State-of-the-Art of Small Spacecraft Technology Avionics,” NASA, 2021. Disponible: https://www.nasa.gov/wp-content/uploads/2021/10/8.soa_avionics_2021.pdf
- [5] Amazon Web Services, “What is FreeRTOS?,” AWS Documentation FreeRTOS User Guide. Disponible: <https://docs.aws.amazon.com/freertos/latest/userguide/what-is-freertos.html>
- [6] Zephyr Project, “Introduction,” Zephyr Project Documentation. Disponible: <https://docs.zephyrproject.org/latest/introduction/index.html>
- [7] R. Kallimani, K. Pai, P. Raghuwanshi, S. Iyer, y O. L. A. López, “TinyML: Tools, applications, challenges, and future research directions,” Multimedia Tools and Applications, vol. 83, n.º 10, pp. 29015–29045, 2024. Disponible: <https://doi.org/10.1007/s11042-023-16740-9>
- [8] Zephyr Project, “Over-the-Air update,” Zephyr Project Documentation. Disponible: https://docs.zephyrproject.org/latest/services/device_mgmt/ota.html
- [9] Arm, “A bare-metal programming guide,” Arm Community, 2023. Disponible: <https://developer.arm.com/community/arm-community-blogs/b/internet-of-things-blog/posts/a-bare-metal-programming-guide>

- [10] European Space Agency (ESA), “CubeSats,” ESA Discovery & Preparation. Disponible: https://www.esa.int/Enabling_Support/Preparing_for_the_Future/Discovery_and_Preparation/CubeSats
- [11] U.S. Food and Drug Administration, “What are examples of Software as a Medical Device?,” FDA.gov. Disponible: <https://www.fda.gov/medical-devices/software-medical-device-samd/what-are-examples-software-medical-device>
- [12] Texas Instruments, “Microcontrollers,” TI.com. Disponible: <https://www.ti.com/product-category/microcontrollers-processors/mcus/overview.html>